SafeTrail — Viva Preparation Guide

Data Structures course project · github.com/adivishall/safetrail

Generated from docs/final/ by tools/build_pdfs.py

Viva questions and answers

Answers are short enough to say aloud and point to where to show the evidence. Earlier review Q&A, mostly about the engine, is in ../REVIEW_QA.md and ../presentation/04_TEACHER_QA.md.

The problem and the product

1. What problem does SafeTrail solve? Tourists in remote areas walk into hazards — cliffs, rivers, closed roads — and help often arrives late because nobody knew where they were or what was happening. SafeTrail tracks people who opt in, checks every position against hazard zones, turns what it finds into alerts and incidents, sends the nearest responder along the road network, keeps working when phones lose signal, and keeps a tamper-evident record of what happened.

2. Why is this a data-structures project and not just an app? Because the core cost is a data-structures problem: every GPS fix asks "which of n zones could contain this point?" Scanning is O(n) per fix; the indexes make it O(log n + k). Around that sit an AVL interval tree for time, a persistent quadtree for history, a heap for routing and triage, union-find for groups and incidents, a hash table, a timer wheel, a ring buffer and a Merkle tree — each doing a job the product needs.

3. Is anything real? The hazard zones and the 144,090-junction road network are real OpenStreetMap data around Shillong. The people are simulated, and every simulated person is labelled SIM. No field trial has been done.

4. How do you know different users don't interfere? Each user owns one `track::Tourist` object holding all their per-zone state, and the evaluator touches only that object. The tests check it from the outside anyway: two users enter the same zone independently, one leaves while the other stays, one stops tracking — each user's events and alerts stay their own (`tests/app/multi_user_test.cpp`).

Indexes

5. Quadtree versus R-tree? A quadtree splits *space* into four fixed quadrants; an R-tree groups the *items* into bounding boxes that may overlap. The quadtree is simpler and its cells never overlap; the R-tree adapts to where data actually is and, bulk loaded with STR, is faster here. Show the Lab's spatial experiment: same query, both trees' nodes, visited and pruned.

6. Why keep brute force? It is the oracle. Every fast index is compared against it on randomised hostile workloads, and the Lab prints the mismatch count for every query.

7. Why does the speedup stop growing at large n in one district? Because about 100 zones genuinely overlap each query; every index must return them. O(log n + k) — the k term dominates. At constant density (k ≈ 1) the speedup keeps growing, to about 1,000× at 100,000 zones.

8. What is STR bulk loading? Sort-Tile-Recursive: sort boxes by longitude, cut into ⌈√P⌉ vertical slices, sort each slice by latitude, pack runs of M into leaves, repeat upward. It produces fewer, fuller, less-overlapping nodes than inserting one at a time: 6.3× faster queries at 100,000 zones in our measurement.

9. What are the worst cases? O(n) for both trees: a quadtree whose boxes all straddle the centre, an R-tree whose envelopes all overlap the query. The bounds are expected, not guaranteed. Removal is O(n) because there is no id → node map.

Trees

10. Why an AVL tree for validity windows? It guarantees height ≤ 1.44 $\log_2(n+2)$, so insert, delete and search are O(log n) in the worst case. Augmenting each node with the maximum end time below it lets a "what was in force at t" query skip whole subtrees.

11. Show me a rotation. Lab → AVL → "Ascending inserts": each step shows the tree, balance factors and the rotations that operation performed. "Zig-zag" shows double rotations.

12. When does the interval tree lose? When the query is not selective: if hundreds of intervals contain every instant, reporting them costs as much as scanning. The benchmark shows it slower than a scan at 1,000 dense intervals. That is why the hot path checks validity in O(1) per candidate and the tree serves history queries only.

13. What is a persistent data structure? One whose old versions stay valid after updates. Path copying copies only the root-to-leaf path an update changes and shares everything else. Here: 1,001 versions need 14,025 nodes instead of 158,257 (11.3×), and a query against the past costs the same as one against the present. Lab → Persistent shows the copied path in amber.

Graphs

14. Dijkstra versus A*? Dijkstra settles junctions in order of distance from the source. A* orders by distance so far plus straight-line distance to the target; because a straight line is never longer than a road, the heuristic is admissible and A* still finds a shortest path while settling fewer junctions. In the demo assignment: 772 versus 5,557.

15. How do you find the nearest responder? One Dijkstra from the incident's junction gives road distance to every responder at once. That is valid because the OSM road graph is undirected; a directed graph would need the search on the reversed graph. Then A* routes the chosen responder.

16. Why a binary heap? Dijkstra needs "next nearest" repeatedly: push and pop in O(log n). Lazy deletion: instead of decreasing a key, push a new entry and skip stale ones on pop.

17. Greedy versus Hungarian? Greedy takes the cheapest pair first and can strand a later incident. Hungarian minimises the total in $O(n^3)$. Measured: 6.5–15.7 % less total travel, never worse.

18. Where is union-find used, and why no path compression? Group cohesion (members within 150 m are joined; more than one component means the group split) and alert correlation (alerts within 200 m and 10 minutes form one incident). Union by rank only, because the structure supports rollback and path compression would change pointers that cannot be undone.

Hashing, queues, time

19. How does your hash table handle collisions and deletions? Linear probing; deletion leaves a tombstone so later keys in the probe chain stay reachable; at 70 % occupancy including tombstones it rebuilds — doubling only if live entries need it. Lab → Hash table shows the probe path.

20. What is the timing wheel for? Escalation: an unacknowledged operator alert escalates after 2 minutes. Each deadline sits in a slot; advancing one tick looks at one slot instead of every alert. Acknowledging cancels the timer.

21. What is the circular buffer for? The last 64 fixes per person, for speed, heading and the stationary and signal-lost detectors. O(1) push, no allocation.

History, evidence, offline

22. How can an examiner see the past? Investigation → pick a time. The trail is binary-searched; zones in force come from the persistent index's version at that time; alert and incident states from their recorded histories. It shows what the system *knew* at that time — events synced later from an offline phone are excluded until they arrived.

23. How do you prove the log was not edited? Every log entry is a leaf of a Merkle tree (RFC 6962). An inclusion proof is O(log n) sibling hashes; anyone with the root can recompute it. The browser verifies proofs itself with WebCrypto; editing an entry changes the root.

24. Why not a blockchain? The requirement is tamper evidence from one authority, verifiable offline. A Merkle log gives that with no network, no consensus and about 200 lines.

25. What happens when a phone loses signal? The phone keeps evaluating zones locally from an index downloaded as bytes, and queues events stamped with a Lamport clock. The server, which has heard nothing, raises "signal lost" after 2 minutes and does not pretend to know where the person is. On reconnect the queue is merged once (a retried upload is rejected event for event), ordered by Lamport stamp, the clock skew is corrected, and late alerts appear marked LATE.

26. Why Lamport clocks and not timestamps? Phone clocks are wrong — the demo phone is 7 minutes fast. Lamport stamps order events consistently with cause and effect without trusting any clock.

Privacy and security

27. Isn't this surveillance? It is designed not to be: tracking only after the person consents and starts a session; nobody else can switch it on; they can stop any time; history expires and can be erased; every look at someone's location by someone else is audited in a tamper-evident log; responders see a person only while assigned to them.

28. How are passwords stored? PBKDF2-HMAC-SHA256 with a random salt and 120,000 iterations, verified against the RFC test vectors. Sessions are random 256-bit tokens in HttpOnly, SameSite=Strict cookies; the server stores only their hash.

Engineering

29. How do you know your tests are any good? Mutation testing: 28 realistic bugs are injected one at a time and a test must fail for each. All 28 are caught.

30. Can it handle many users? On one core it ingests about 300,000 fixes per second, and the cost per fix is flat from 10 to 5,000 users. That benchmark also caught a real bug: alert bookkeeping was quadratic until alerts were indexed per user.

31. What would you do differently or next? Persistence (the server is in memory), real GPS traces, a phone app running the same offline evaluator, TLS and Argon2, and O(log n) index removal. See LIMITATIONS_AND_FUTURE_SCOPE.md.

32. What was the hardest bug? The evaluator only observed zones the index returned, so someone who left a zone's candidate window in one step (a GPS gap) never got an exit and later re-entries were silent. Fixed by reconciling every zone with open state every tick — and pinned by a test and a mutant (../DEFECT_LOG.md).

Syllabus mapping — which Data Structures concepts SafeTrail demonstrates

Every row names the file that implements the concept and where SafeTrail uses it. A concept is listed only if the code really does it; "could be related" is not enough. The **Lab** column says which Data Structures Lab experiment in the web app (`make app` → Data Structures Lab) shows it running on the real C++ code.

Four groups:

- **A. Direct syllabus applications** — core course topics, implemented by hand here.
- **B. Supporting concepts** — course topics used along the way, often through the C++ standard library.
- **C. Beyond the syllabus** — advanced material; ADVANCED_CONCEPTS.md explains each.
- **D. Not implemented** — listed so nobody claims them.

Each concept is explained twice: once in plain words, once technically, with a SafeTrail example.

A. Direct syllabus applications

Syllabus concept	Implementation	File / function	Where SafeTrail uses it	Complexity	Lab
Arrays / dynamic arrays	`std::vector` as the backing store of every structure; flat `ZoneStore` indexed by id	`fence::ZoneStore` (`src/fence/zone.cpp`)	zone storage, alert table, trails	O(1) index, amortised O(1) append	—
Circular (ring) buffer	fixed array + wrapping head	`ds::CircularBuffer` (`include/safetrail/ds/circular_buffer.hpp`)	last 64 GPS fixes per person	O(1) push and index	Circular buffer
Linked (pointer) structures	tree nodes owned by `unique_ptr` / `shared_ptr`	`index::RTree::Node`, `index::Quadtree::Node`, persistent `VersionedIndex::Node`	the spatial indexes	—	Spatial; Persistent
Stack	explicit stack in an iterative DFS	`ds::components_bruteforce` (`src/ds/dynamic_connectivity.cpp`)	test oracle for union-find	O(V + E)	—

Syllabus concept	Implementation	File / function	Where SafeTrail uses it	Complexity	Lab
Queue (priority)	binary min/max heap	`ds::BinaryHeap` (`include/safetrail/ds/priority_queue.hpp`)	Dijkstra and A* frontier; alert triage	push/pop O(log n), top O(1)	Binary heap; Graph
Trees (general), recursion	recursive build, query, delete	quadtree and R-tree `query_node`, `insert_into`, `remove_node`	every geofence query	O(depth) recursion	Spatial
Binary search tree	BST ordered on (low, high, insertion seq)	`ds::IntervalTree` (`include/safetrail/ds/interval_tree.hpp`)	zone validity history	O(log n) search	AVL
AVL tree, rotations	height-balanced BST, single and double rotations, rebalance on delete	`IntervalTree::rot_left/rot_right/rebalance`	as above	height ≤ 1.44 $\log_2(n+2)$; insert/delete O(log n)	AVL (shows each rotation)
AVL tree (second instance)	index-backed AVL tree as the sweep-line status structure	`src/geo/sweep_line.cpp`	rejecting self-intersecting zone polygons (`Polygon::validate`)	O(V log V) per polygon	—
Augmented tree	each node stores max end time of its subtree	`IntervalTree::refit`, prune in `descend`	"which rules were in force at t"	stab O(min(n, (k+1) log n))	AVL (shows nodes visited)
Tree traversal	pre-order (describe), in-order (layout), DFS queries	`Quadtree::describe`, `IntervalTree::collect_view`	Lab, history queries	O(n)	AVL, Spatial
Quadtree	region quadtree over boxes, fitted root, collapse on delete	`index::Quadtree` (`src/index/quadtree.cpp`)	spatial index option	query O(log n + k) expected	Spatial
R-tree	Guttman quadratic split; STR bulk loading; condensing delete	`index::RTree` (`src/index/rtree.cpp`)	the platform's live spatial index	query O(log n + k) expected	Spatial (STR vs insert)
k-d tree	2-d tree, median split, nearest neighbour with pruning	`index::KdTree` (`include/safetrail/index/kd_tree.hpp`)	snapping to road junctions	build O(n log n), NN O(log n) expected	k-d tree

Syllabus concept	Implementation	File / function	Where SafeTrail uses it	Complexity	Lab
Hashing, hash table	open addressing, linear probing, tombstones, load-factor rehash	`ds::HashMap` (`include/safetrail/ds/hash_table.hpp`)	usernames, sessions, escalation records, sync de-duplication	O(1) expected	Hash table
Collision handling	linear probing; deletion by tombstone	`HashMap::put/find/erase/maybe_rehash`	as above	probe length grows with load	Hash table (shows probe)
Graphs, adjacency list	`std::vector<std::vector<Edge>>`	`graph::RoadGraph` (`include/safetrail/graph/road_graph.hpp`)	144,090-junction OSM road network	O(V + E) space	Graph
Shortest paths: Dijkstra	binary-heap Dijkstra with lazy deletion	`graph::dijkstra` (`src/graph/dijkstra.cpp`)	nearest responder by road distance	O((V + E) log V)	Graph
A* search	Dijkstra + admissible straight-line heuristic	`graph::astar` (`src/graph/astar.cpp`)	responder route	≤ Dijkstra's work	Graph
Disjoint-set union (union-find)	union by rank, rollback stack	`ds::RollbackDSU` (`src/ds/dynamic_connectivity.cpp`)	group cohesion; alert → incident correlation	O(log n) per op	Union-find
Searching: binary search	`upper_bound` on a time-sorted trail; sorted geohash keys	`Platform::position_at` (`src/app/platform.cpp`), `index::Geohash`	"where was X at time t"	O(log n)	Investigation page
Algorithmic complexity, worst vs expected	brute force kept as the O(n) baseline; bounds stated per structure	`index::BruteForceIndex`; ALGORITHM_COMPLEXITY.md	every comparison	—	Spatial (timings)
Empirical performance measurement	interleaved, paired, repeated benchmarks	`apps/safetrail_bench.cpp` (19 sections)	`make bench`, Analytics page	—	—

B. Supporting concepts

Concept	Where	Notes
Sorting	`std::sort` / `std::stable_sort` in STR packing (`src/index/rtree.cpp`), Lamport merge (`src/sync/lamport.cpp`), incident ranking (`alert::rank_by_seriousness`); `std::nth_element` (median selection) in the k-d tree build	Library sorts are used; none is hand-written. The **stability** and **total order** requirements are worked out in the code: incident ranking needed an id tie-break because `std::sort` is unstable.
Selection (median)	`std::nth_element` in `KdTree::build_rec`	expected O(n) per level → O(n log n) build
Recursion vs iteration	recursive tree code; iterative DFS in the union-find oracle; iterative Dijkstra	
Amortised analysis	`std::vector` growth; hash table rehash; Merkle log append	the hash table rehashes by doubling **only** if live entries need it, else rebuilds at the same size to drop tombstones
Time–space trade-offs	persistent index (shares nodes instead of copying); per-zone validity checked in O(1) on the hot path but stored in an interval tree for history	ALGORITHM_COMPLEXITY.md §6
Bipartite matching	Kuhn's augmenting paths (`graph::kuhn_matching`) and the Hungarian algorithm (`graph::hungarian`)	`src/graph/bipartite_match.cpp`; dispatch
State machines	hysteresis filter (`fence::HysteresisState`), alert and incident life cycles	
Computational geometry	ray casting, winding number, segment intersection, Shamos–Hoey sweep line (with a balanced status tree)	`src/geo/`
Differential testing	every index against brute force, with invariant audits after every operation	`tests/index/differential_test.cpp`
Mutation testing	23 injected bugs, each must fail a test	`tools/mutation_check.py`

C. Beyond the syllabus (detail in ADVANCED_CONCEPTS.md)

Persistent data structures and structural sharing · STR bulk loading · Z-order (Morton) curves · hashed timing wheel · Merkle trees with RFC 6962 inclusion proofs · SHA-256, HMAC, PBKDF2 · Lamport logical clocks and offline reconciliation · bitemporal history (when it happened vs when the system knew) · hysteresis and three-valued containment under GPS uncertainty · predictive crossing · Hungarian assignment · battery-aware adaptive sampling · single-writer event loop with `poll(2)` · deterministic replay across compilers.

D. Not implemented (do not claim)

Topic	Status
Breadth-first search	no BFS in the product; components come from union-find (the test oracle uses a DFS with an explicit stack)
Linked lists as a data structure	trees use pointers, but there is no linked list
Hand-written sorting algorithms (quick/merge/heap sort)	library sorts only; the heap exists as a priority queue, not as heapsort
B-trees / B+ trees, tries, skip lists, red-black trees	absent
Minimum spanning trees, topological sort, strongly connected components	absent
Dynamic programming	absent
String algorithms	absent

Every concept, explained twice

Dynamic array (`std::vector`)

Simply: a row of boxes that grows when it fills, by moving into a bigger row. **Technically:** contiguous storage with capacity ≥ size; appends are amortised O(1) because capacity doubles, so n appends copy O(n) elements in total. **SafeTrail:** the zone store is a vector indexed by zone id, so `get(id)` is one array access.

Circular buffer

Simply: a clock face with 64 slots; each new GPS fix goes in the next slot and overwrites the oldest. **Technically:** `head = (head + 1) mod N`, `count = min(count + 1, N)`; element i (newest first) is at `(head − 1 − i) mod N`. O(1) everything, no allocation. **SafeTrail:** speed, heading and the "stationary" and "signal lost" checks read the last 64 fixes.

Stack

Simply: a pile of plates — take from the top. **Technically:** LIFO; here a vector used with `push_back` / `pop_back` to run a depth-first search without recursion. **SafeTrail:** the union-find test oracle labels connected components by DFS.

Binary heap / priority queue

Simply: a tournament bracket in which the winner is always at the top. **Technically:** a complete binary tree in an array; children of i at 2i+1 and 2i+2; parent ≤ children. Push sifts up, pop moves the last leaf to the root and sifts down: O(log n). **SafeTrail:** Dijkstra pops the nearest unsettled junction; the alert board pops the most urgent alert, where urgency = severity + 0.5 per minute open + 0.1 per km to the nearest free responder.

Tree, recursion and traversal

Simply: a family tree; to search it you ask each child to search its own branch. **Technically:** recursive descent with pruning — a quadtree query returns at once from any node whose box misses the query box, so whole subtrees are skipped. **SafeTrail:** every GPS fix runs one such query.

Binary search tree and AVL tree

Simply: a sorted tree that keeps itself evenly shaped by "rotating" lopsided parts. **Technically:** BST order on (start, end, insertion number); AVL keeps $|\text{height(left)} - \text{height(right)}| \le 1$ at every node by single or double rotations after inserts and deletes, so height $\le 1.44 \log_2(n + 2)$. Deletion can need a rotation at every level on the way up, so rebalancing is general rather than insert-specific. **SafeTrail:** holds every zone's validity windows over all history.

Augmented tree (interval tree)

Simply: each branch remembers the latest time anything under it ends, so a question about 14:32 can skip branches that ended at 14:00. **Technically:** `max_high = max(own end, left.max_high, right.max_high)`, maintained by rotations; a stabbing query prunes any subtree with `max_high ≤ t`. Bound O(min(n, (k + 1) log n)). **SafeTrail:** "which zones were in force at t?" in the persistent index.

Quadtree

Simply: cut the map into four squares, and keep cutting any square that holds too many zones. **Technically:** region quadtree over bounding boxes; root fitted to the data and doubled outward when a zone falls outside; a box straddling a split stays at the internal node; subtrees collapse after deletes. **SafeTrail:** an index option; the Lab shows its cells and which ones a query entered.

R-tree and STR packing

Simply: instead of cutting the map, put nearby zones into labelled envelopes, and envelopes into bigger envelopes. **Technically:** height-balanced tree of minimum bounding rectangles (MBRs), node capacity 8; quadratic split on overflow (Guttman); **STR bulk load** sorts by longitude into $\sqrt{(n/M)}$ vertical slices, sorts each slice by latitude, and packs runs of M — fuller nodes, less overlap. **SafeTrail:** the platform's live index; its measured speedup over brute force at 100,000 zones is in RESULTS.md §1 (regenerated by `make bench`).

k-d tree

Simply: split the points alternately left/right and up/down at the middle one. **Technically:** median split by `nth_element` on alternating axes; nearest-neighbour search descends towards the query, then crosses a split only if the splitting plane is closer than the best point found. **SafeTrail:** snaps a responder or incident to the nearest of 144,090 road junctions.

Hash table with linear probing

Simply: a key's number tells you which drawer to try; if it is taken, try the next one. **Technically:** Fibonacci hashing `k × 2654435769`, masked to a power-of-two table; linear probing; deletion leaves a tombstone so probe chains do not break; rehash when (live + tombstones) reach 70 %, growing only if the live entries need it. **SafeTrail:** the username index, sessions (keyed by SHA-256 of the token), alert escalation records, offline-sync de-duplication.

Graph, adjacency list

Simply: each junction keeps a list of the roads leaving it. **Technically:** `vector<vector<Edge{to, weight}>>`, O(V + E) space, neighbours in O(degree). **SafeTrail:** the OpenStreetMap road network.

Dijkstra and A*

Simply: Dijkstra grows a circle of "known shortest" junctions outward; A* grows it towards the target first, guided by straight-line distance. **Technically:** Dijkstra settles nodes in increasing distance from a binary heap (lazy deletion: stale entries are skipped on pop). A* orders by f = g + h with h = great-circle distance, which never exceeds road distance (admissible), so the first time the target is popped its cost is optimal. **SafeTrail:** one Dijkstra from an incident gives the road distance to every responder (valid because the road graph is undirected); A* then routes the chosen one. Assigning an incident in the walkthrough, A* settled 772 junctions where Dijkstra settles 5,557 for the same pair.

Union-find (disjoint-set union) with rollback

Simply: everyone points to a group leader; to merge two groups, one leader points to the other. **Technically:** union by rank keeps trees O(log n) deep; **no path compression**, because rollback must be able to restore every pointer exactly; each union is recorded on an undo stack. **SafeTrail:** whether a tour group is still one group (members within 150 m are linked); which alerts are one incident (within 200 m and 10 minutes).

Binary search

Simply: to find a time in a sorted list, look in the middle and throw away half. **Technically:** `std::upper_bound` over the time-sorted trail, O(log n). **SafeTrail:** "where was this person at 10:05?" in the Investigation page.

Complexity, worst versus expected, measured

Simply: big-O says how the work grows; benchmarks say what it actually costs on this laptop. **Technically:** spatial trees are O(log n + k) *expected* and O(n) *worst case*; the AVL tree's height bound is *guaranteed*; constants and cache effects decide real times, which is why every claim here has a measurement next to it. **SafeTrail:** the speedup stops growing once k (zones that really overlap the query) dominates — measured in section 1 and explained in TESTING_AND_BENCHMARKS.md.

Concepts beyond the Data Structures syllabus

Each concept below is something the code really does. For each: what it is, why it goes beyond a first data-structures course, the practical problem it solves here, and where to find it.

Concept	Where	Problem it solves
Computational geometry	`src/geo/`	is a fix inside a polygon; is a polygon valid
Coordinate systems and projections	`src/geo/projection.cpp`, `geo::distance_m`, Web Mercator in the web map	distances in metres from degrees
GPS uncertainty modelling	`geo::Containment`, `sim::GpsErrorModel`	a fix is a disc, not a point
Hysteresis	`src/fence/hysteresis.cpp`	drift should not flap a boundary
State machines	hysteresis phases; alert and incident life cycles	each transition reported once
Persistent data structures	`index::VersionedIndex`	every past version, cheaply
Temporal (bitemporal) history	`TrailPoint::recorded_ms`, `Platform::position_at`	what happened vs what the system knew
Lamport clocks, offline-first reconciliation	`sync/`, `Platform::go_offline/reconnect`	ordering events from phones with wrong clocks
Cryptographic integrity	SHA-256, Merkle proofs (`evidence/`)	proving the log was not edited
Password hashing and session security	`app/crypto.cpp`, `app/api.cpp`	accounts that resist offline guessing and CSRF
Event-driven, single-writer architecture	`net::Server::run`, `app::Platform`	one total order, no locks
Deterministic replay	`-ffp-contract=off`, seeded RNGs, total-order tie breaks	the same run gives byte-identical evidence
Spatio-temporal correlation	`alert::Correlator`	forty alerts at one landslide are one incident
Battery-aware computation	`power::AdaptiveSampler`	sample less often far from hazards
Performance engineering and benchmark methodology	`apps/safetrail_bench.cpp`	numbers that survive scrutiny
Differential and mutation testing	`tests/index/differential_test.cpp`, `tools/mutation_check.py`	evidence that the tests can catch bugs
Numerical tolerance	`geo::` on-edge tolerance, `-ffp-contract=off`	floating point flips inside/outside

Computational geometry

What: ray casting (count crossings of a ray with polygon edges; odd = inside), the winding number (an independent second answer used as an oracle), segment intersection with orientation tests, and the Shamos–Hoey sweep line that detects a self-intersecting polygon in O(V log V) with its own AVL status tree. **Why beyond the syllabus:** it needs robustness reasoning — collinear and touching edges, vertices exactly on the ray, holes. **Here:** every candidate zone is tested exactly after the index prunes; zone creation and import refuse self-intersecting polygons, so the store never holds geometry that would make ray casting return arbitrary results. ../GEOMETRY_EDGE_CASES.md lists the edge cases and tolerances.

Three-valued containment under GPS uncertainty

What: a fix with accuracy r is a disc. If the whole disc is inside → *inside*; wholly outside → *outside*; straddling → *uncertain*. **Here:** `geo::evaluate(poly, fix)` returns all three; the platform raises *uncertain* warnings for serious zones. A real finding from the OSM data: the river and cliff polygons are only 23–40 m deep, so with a 15 m entry margin and 5–18 m fixes "uncertain" is often the honest answer.

Hysteresis and state machines

What: a Schmitt trigger — enter only 15 m inside, exit only 25 m outside, three agreeing fixes, a 5 s minimum dwell; an *ambiguous* phase for uncertain readings. **Problem:** GPS drift near a boundary otherwise produces an enter/exit pair every few seconds. The A/B benchmark (section 10) measures the filter removing the flood while keeping the real transitions.

Persistent data structures and structural sharing

What: data structures whose old versions remain valid after updates (Driscoll, Sarnak, Sleator and Tarjan, 1989). Path copying shares every untouched node. **Here:** each zone change is a new version of the quadtree; the sharing ratio is measured in section 9 and shown live on the Analytics page.

Bitemporal history

What: two times per fact — when it happened (valid time) and when the system recorded it (transaction time). **Here:** a fix synced from an offline phone happened at 10:02 but arrived at 10:12. The Investigation page answers "what did the system know at 10:05?" using both, so it never shows knowledge the system did not yet have (`tests/app/scenario_test.cpp` asserts this).

Lamport logical clocks and offline reconciliation

What: counters that respect causality without trusting wall clocks (Lamport, 1978). Local event: counter + 1. Receive: max(own, received) + 1. Order: (counter, device). **Here:** an offline phone keeps evaluating zones locally from an index shipped as bytes, queues Lamport-stamped events, and on reconnect the server's reconciler admits each event once (retries are rejected by a hash-set lookup), orders by Lamport stamp, estimates and corrects the clock skew, and adopts the phone's engine state so no transition is repeated. The offline scenario demonstrates a phone 7 minutes fast. **Finding:** a phone with a wrong clock judges zone validity windows on that wrong clock — recorded as a limitation.

Cryptographic integrity: SHA-256 and Merkle proofs

What: a Merkle tree over an append-only log (RFC 6962): any entry can be proven present with O(log n) hashes against a published root; editing any entry changes the root. **Here:** every platform log entry is a leaf. Entries carry ids, never coordinates, so erasing a person's movement history (a privacy right) never conflicts with the evidence log. Verified independently in the browser (WebCrypto) and in `tools/api_e2e.py` (Python hashlib).

Password hashing and session security

What: PBKDF2-HMAC-SHA256 (RFC 8018) with a random salt and 120,000 iterations; HMAC per RFC 2104; random 256-bit session tokens from the OS; only the SHA-256 of a token is stored; constant-time comparisons. Checked against the RFC test vectors. **Why beyond:** security engineering, not data structures — but it uses the project's own SHA-256.

Single-writer event loop

What: one thread multiplexes sockets with `poll(2)` and interleaves the simulation clock. Every mutation happens in one total order with no locks. **Trade-off:** no parallelism; measured throughput is still about 300,000 fixes per second on one core (section 19).

Deterministic replay

What: the same seed must give byte-identical output on Apple clang and Linux g++. Fused multiply-add contraction is disabled (`-ffp-contract=off`); every sort that a cap cuts through has a total-order tie-break; scenarios are seeded. **Here:** CI compares the dashboard built on macOS and Linux byte for byte; `tests/app/scenario_test.cpp` asserts identical event logs and Merkle roots across runs.

Spatio-temporal correlation

What: connected components of alerts under "within 200 m and 10 minutes" (single linkage), via union-find; new alerts near an open incident join it. **Here:** a breach and its dwell overrun are one incident; an injured hiker's stationary and straggler alerts are one incident.

Battery-aware computation

What: the sampling interval depends on distance to the nearest zone (2 s under 200 m … 5 min beyond 5 km), forced to full rate while a serious alert is open. **Here:** the evaluator feeds the sampler the distance it already computed; a battery-aware device in the platform sends fixes only when due, and the signal-lost detector stretches its threshold to 2.5× the expected interval so silence by design is not mistaken for a lost signal. The energy figure is a stated linear model, not a measurement.

Benchmark methodology, differential and mutation testing

See TESTING_AND_BENCHMARKS.md: interleaved rounds, paired ratios, checksums as correctness gates, oracles for every fast structure, invariant audits after every operation, and 23 injected bugs that the suite must catch.

Presentation script (14 slides; about 8 minutes, with the live demo after slide 11)

Slide numbers match `docs/final/SafeTrail_final_presentation.pptx`. Speaker notes in the deck carry the same text.

1. Title — SafeTrail. "SafeTrail is a geofencing and incident-response system for tourist safety in low-connectivity regions, built on hand-written data structures in C++17."

2. The problem. "In the hills around Shillong, cliffs, rivers and closed roads are where people get hurt, and phones lose signal exactly there. Help is late when nobody knows who is where. Checking every position against every hazard zone is O(n) per GPS fix — the cost the whole design is about."

3. What it does. "People opt in. Every fix is checked against hazard zones, with GPS uncertainty treated as a disc, not a point. Transitions become alerts, related alerts become incidents, the nearest responder is routed along roads, groups that split are noticed, phones that go offline keep working and sync later, and everything is kept in a tamper-evident history."

4. Architecture. "Three layers: the engine — indexes, geometry, the evaluator; the platform — users, consent, sessions, alerts, incidents, groups, responders, history, offline sync; and a single-threaded HTTP server with a web console. No database, no map service, no dependencies."

5. The core structures. "Brute force as the oracle; a quadtree that splits space; an R-tree that groups items, bulk-loaded with STR; an AVL interval tree for validity windows; a persistent quadtree for every past version."

6. Supporting structures. "A binary heap for Dijkstra, A* and alert triage; union-find with rollback for groups and incidents; an open-addressing hash table for users, sessions and sync; a timer wheel for escalation; a ring buffer for recent fixes; a k-d tree for snapping to the road network; a Merkle tree for evidence."

7. Results. "At 100,000 zones in one district the R-tree answers 224 times faster than a scan, the quadtree 33 times — identical answers. At constant density, where only about one zone overlaps each query, the speedups reach about 550× and 1,000×. STR packing makes the R-tree 6× faster than building it by insertion; A* settles about 75 % fewer junctions than Dijkstra."

8. Multi-user. "Through the whole platform a fix costs about 3 microseconds, and that cost does not change from 10 users to 5,000 — about 300,000 fixes a second on one core, roughly 7 KB per tracked person. The same benchmark found a real bug: alert bookkeeping was quadratic until alerts were indexed per user."

9. Correctness. "Every fast structure has an oracle; invariants are checked after every operation; 51 test files, 12,137 checks; 28 injected bugs, all caught; byte-identical output on macOS and Linux."

10. Privacy. "Opt-in only, nobody else can switch it on, stop any time, history expires and can be erased, every look at someone's location is audited, responders see a person only while assigned."

11. Live demo — switch to the browser (LIVE_DEMO_SCRIPT.md).

12. Syllabus map. "Arrays and ring buffers, stacks, trees and recursion, BSTs and AVL rotations, augmented trees, hashing with collision handling, heaps and priority queues, graphs and adjacency lists, Dijkstra and A*, union-find, binary search, complexity and measurement. Beyond: persistence, Merkle proofs, Lamport clocks, computational geometry."

13. Limitations and future. "Simulated people; an in-memory server that runs on this laptop — the public link is only a static replay; the OSM hazard polygons are thin, so 'uncertain' is often the honest answer; benchmarks from one laptop. Next: persistence, real GPS traces, a phone app running the same offline evaluator, public hosting behind TLS, and Argon2."

14. Close. "Thank you — the code, tests, benchmarks and this deck are in the repository, and every benchmark figure on these slides is regenerated by `make bench`."

Live demo script (7–10 minutes)

Exact actions, what should appear, and what to do if it does not. Rehearse it twice from a clean start. Times in **[brackets]** are cumulative targets.

Before the examiner arrives (5 minutes)

```
cd ~/Projects/"Data Structures CP" && git switch final/product-completion && make app-reset && make app
```

Expected terminal output:

```
SafeTrail demo mode — http://127.0.0.1:8080  (world built in 0.4 s: 38 zones, 144090 road junctions)
Accounts (passwords from the environment or var/demo-credentials.txt):
  admin      admin     <generated>
  operator   operator  <generated>
  ...
Scenario "walkthrough" is loaded and paused at 00:00 — press Start in Live Monitoring.
```

1. Open **http://127.0.0.1:8080** in Chrome, full screen, at 100 % zoom.
2. Sign in as **admin** (password from the terminal, or `cat var/demo-credentials.txt`).
3. In a second terminal, have `make test` already run once (so it finishes in seconds when shown).
4. Keep `docs/presentation/SafeTrail_review_slides.pdf` or the new deck open in another window as the fallback.

The demo

#	Say (short)	Do	Expect
1 **[0:00]**	"Tourists in remote areas walk into hazards; help arrives late. SafeTrail tracks only people who opt in, and turns positions into alerts, incidents and dispatch."	Show the Overview page.	KPI strip, map with 38 OSM zones, 6 DEMO responders, scenario panel.
2 **[0:45]**	"Architecture: a C++17 engine with hand-written structures; a platform layer for users, consent and incidents; this single-threaded server; no external services."	One slide (architecture), back to browser.	—
3 **[1:30]**	"Everyone you will see is simulated and marked SIM; the zones and roads are real."	Top bar: speed **10×**, press **Run**. Go to **Live Monitoring**.	Clock advancing; 16 markers appear as each tourist opts in; group labels.
4 **[2:15]**	"Each user has independent state; every fix goes through the same endpoint a phone uses."	Click one tourist (e.g. Anika) on the map.	Selected tab: ID, coordinates, last update, accuracy, nearest zones, transitions, battery model.

#	Say (short)	Do	Expect
5 **[3:00]**	"Ravi walks toward a restricted closure the administrator registered."	Watch Waterfall Viewpoint (zoom with the wheel). Scenario tab shows the narration.	Around T+00:05:50: approaching (amber), then uncertain at the edge, then **inside restricted** (red) and an incident ring **#1**. Alerts badge increases.
6 **[4:00]**	"One breach → alert → incident, with the nearest responder by road distance — one Dijkstra from the incident."	**Pause.** Go to **Incidents**, click Ravi's incident.	Recommended: a Demo unit and its road distance.
7 **[4:30]**	"Acknowledge, assign: the route is A* on the OpenStreetMap graph."	**Acknowledge**, then **Assign**.	State *dispatched*; white route on the mini-map; "A* settled ~770 junctions; Dijkstra needs ~5,500".
8 **[5:00]**	"Group separation: Nisha stopped; union-find over the group's proximity graph says the group split; after 60 s it is a Straggler alert."	**Groups** (or Live map near Ridge Hikers); **Run** to ≈ T+00:10.	Ridge Hikers "split into 2"; dashed line to Nisha with the separation; her incident has Stationary + Straggler alerts.
9 **[5:45]**	"Offline: Tenzin's phone kept evaluating locally; on reconnect the queue is merged once by Lamport order."	Live → **Scenario** tab after ≈ T+00:12:30.	Reconciliation report: N uploaded, N admitted, N duplicates rejected, skew corrected, late alert raised; Alerts shows **LATE**.
10 **[6:15]**	"Investigation: what did the system know at a past time? And is the log intact?"	**Investigation** → drag the slider to ≈ T+00:06:00 → Query; scroll to the event log → **Verify** on a row.	Positions known then; zones in force from the persistent index's version; "✓ verified" — recomputed in the browser.
11 **[7:00]**	"The Data Structures Lab runs the real C++ structures."	**Data Structures Lab** → **Brute force · Quadtree · R-tree**; drag a new query box; press Play.	Teal (descended) and red (pruned) nodes; table: every index 0 mismatches vs brute force; timings.
12 **[7:45]**	"AVL rotations and graph search, from the same code."	Lab → **AVL** (Play), then **Graph** (switch Dijkstra / A*).	Balance factors, rotations; settled nodes and heap contents.
13 **[8:15]**	"Benchmarks are measured, not typed."	**Analytics**, scroll to the charts.	Index scaling chart; multi-user table (≈ 3 µs per fix up to 5,000 users).
14 **[8:45]**	"Tests: 51 files, 12,137 checks; 28 injected bugs all caught."	Second terminal: `make test` (and point at the last line).	`ALL TESTS PASS`.
15 **[9:15]**	"Syllabus: arrays and ring buffers, trees and recursion, BST and AVL, hashing, heaps, graphs, Dijkstra/A*, union-find, binary search; beyond it: persistence, Merkle proofs, Lamport clocks. Limits: simulated people, in-memory server, thin OSM polygons."	Slide: syllabus map + limitations.	—

Fallbacks

Problem	Fallback
Port 8080 busy	`make app APP_ARGS="--port 8090"`
The scenario has run past the moment you need	top bar **Reset**, then **Step 10 s** / speed 60× to jump
Incident list refreshes under the cursor	click the incident in the Overview table instead, or pause first
Browser problem	the static dashboard: `make dashboard` then open `dashboard.html`, or the live site https://adivishall.github.io/safetrail
Laptop problem	the PDF report and deck in `docs/final/`, and the screenshots in `docs/final/images/`
Lab request error	any experiment's **Run/Build** button resubmits; the server log shows nothing sensitive
Forgotten password	`cat var/demo-credentials.txt`

Clean-start check (run once before the day)

```
make app-reset && make -s && make test && make e2e
```

Expected: `ALL TESTS PASS` and `[ ok ] api/e2e 30 checks, 0 failed` (about 22 s).